

```

import serial
import serial.tools.list_ports
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import threading
import time
from datetime import datetime
import csv

class SimpleRobotControl:
    def __init__(self, root):
        self.root = root
        self.root.title("Robot vestibular - comunicación")
        self.root.geometry("700x700")

        # Variables de estado
        self.ser = None
        self.connected = False
        self.recording = False
        self.csv_file = None
        self.csv_writer = None
        self.sample_count = 0
        self.start_timestamp = 0
        self.csv_header = None # Variable para almacenar el header

        # Configurar interfaz
        self.setup_ui()

    def setup_ui(self):
        # =====
        # SECCIÓN 1: CONEXIÓN SERIAL
        # =====
        frame_conexion = ttk.LabelFrame(self.root, text="Conexión Serial", padding="10")
        frame_conexion.pack(fill="x", padx=10, pady=10)

        # Puerto COM
        ttk.Label(frame_conexion, text="Puerto COM:").grid(row=0, column=0, sticky="w", padx=5)
        self.port_combo = ttk.Combobox(frame_conexion, width=15)
        self.port_combo.grid(row=0, column=1, padx=5)

        # Baudrate
        ttk.Label(frame_conexion, text="Baudrate:").grid(row=0, column=2, sticky="w", padx=5)
        self.baud_combo = ttk.Combobox(frame_conexion, width=10, values=["9600", "115200",
"250000"])
        self.baud_combo.grid(row=0, column=3, padx=5)
        self.baud_combo.set("115200")

        # Botones de conexión
        self.btn_refresh = ttk.Button(frame_conexion, text="Refrescar", command=self.refresh_ports)
        self.btn_refresh.grid(row=0, column=4, padx=5)

        self.btn_connect = ttk.Button(frame_conexion, text="Conectar",
command=self.toggle_connection)

```

```

self.btn_connect.grid(row=0, column=5, padx=5)

# Estado de conexión
self.status_label = ttk.Label(frame_conexion, text="Desconectado", foreground="red")
self.status_label.grid(row=1, column=0, columnspan=6, pady=(10, 0), sticky="w")

# =====
# SECCIÓN 2: CONTROL DE PARÁMETROS
# =====
frame_control = ttk.LabelFrame(self.root, text="Control de Posición", padding="10")
frame_control.pack(fill="x", padx=10, pady=10)

# Motor 1
ttk.Label(frame_control, text="Motor 1 (grados):").grid(row=0, column=0, sticky="w", pady=5)
self.entry_m1 = ttk.Entry(frame_control, width=10)
self.entry_m1.grid(row=0, column=1, padx=5, pady=5)
self.entry_m1.insert(0, "1000")
ttk.Button(frame_control, text="Enviar M1", command=self.send_m1).grid(row=0, column=2,
padx=5)

# Motor 2
ttk.Label(frame_control, text="Motor 2 (grados):").grid(row=1, column=0, sticky="w", pady=5)
self.entry_m2 = ttk.Entry(frame_control, width=10)
self.entry_m2.grid(row=1, column=1, padx=5, pady=5)
self.entry_m2.insert(0, "1000")
ttk.Button(frame_control, text="Enviar M2", command=self.send_m2).grid(row=1, column=2,
padx=5)

# Servo SSS
ttk.Label(frame_control, text="Servo SSS (10-170):").grid(row=2, column=0, sticky="w",
pady=5)
self.entry_sss = ttk.Entry(frame_control, width=10)
self.entry_sss.grid(row=2, column=1, padx=5, pady=5)
self.entry_sss.insert(0, "105") #PARÁMETRO INICIALES (COMPENSADOS)
ttk.Button(frame_control, text="Enviar SSS", command=self.send_sss).grid(row=2, column=2,
padx=5)

# Servo SIS
ttk.Label(frame_control, text="Servo SIS (10-170):").grid(row=3, column=0, sticky="w", pady=5)
self.entry_sis = ttk.Entry(frame_control, width=10)
self.entry_sis.grid(row=3, column=1, padx=5, pady=5)
self.entry_sis.insert(0, "100") #PARÁMETROS COMPENSADOS
ttk.Button(frame_control, text="Enviar SIS", command=self.send_sis).grid(row=3, column=2,
padx=5)

# =====
# SECCIÓN 3: ADQUISICIÓN DE DATOS
# =====
frame_data = ttk.LabelFrame(self.root, text="Adquisición de Datos", padding="10")
frame_data.pack(fill="x", padx=10, pady=10)

# Checkbox para timestamp relativo
self.ts_relativo_var = tk.BooleanVar(value=True)

```

```

self.check_ts = ttk.Checkbutton(frame_data, text="Timestamp relativo (empezar desde 0)",
                                variable=self.ts_relativo_var)
self.check_ts.pack(anchor="w", pady=(0, 10))

# Botones de control de adquisición
btn_frame = ttk.Frame(frame_data)
btn_frame.pack(fill="x")

self.btn_start = ttk.Button(btn_frame, text="Iniciar Grabación",
                             command=self.start_recording, state="disabled")
self.btn_start.pack(side="left", padx=5)

self.btn_stop = ttk.Button(btn_frame, text="Detener Grabación",
                             command=self.stop_recording, state="disabled")
self.btn_stop.pack(side="left", padx=5)

# Información de estado
self.recording_label = ttk.Label(btn_frame, text="No grabando", foreground="red")
self.recording_label.pack(side="left", padx=20)

# Contador de muestras
self.sample_label = ttk.Label(btn_frame, text="Muestras: 0")
self.sample_label.pack(side="right", padx=5)

# =====
# SECCIÓN 4: CONSOLA DE MENSAJES
# =====
frame_console = ttk.LabelFrame(self.root, text="Consola", padding="10")
frame_console.pack(fill="both", expand=True, padx=10, pady=10)

# Consola de texto
self.console = tk.Text(frame_console, height=8, width=60)
self.console.pack(fill="both", expand=True)

# Barra de desplazamiento
scrollbar = ttk.Scrollbar(self.console)
scrollbar.pack(side="right", fill="y")
self.console.config(yscrollcommand=scrollbar.set)
scrollbar.config(command=self.console.yview)

# Inicializar
self.refresh_ports()

# Iniciar thread para lectura serial
self.serial_thread = threading.Thread(target=self.read_serial, daemon=True)
self.serial_thread.start()

# =====
# FUNCIÓN: Refrescar puertos COM disponibles
# =====
def refresh_ports(self):
    """Listar puertos COM disponibles en el sistema"""
    ports = [port.device for port in serial.tools.list_ports.comports()]

```

```

self.port_combo['values'] = ports
if ports:
    self.port_combo.set(ports[0])

# =====
# FUNCIÓN: Conectar/Desconectar Arduino
# =====
def toggle_connection(self):
    """Establecer o cerrar conexión serial con Arduino"""
    if not self.connected:
        self.connect_serial()
    else:
        self.disconnect_serial()

def connect_serial(self):
    """Conectar al puerto serial seleccionado"""
    port = self.port_combo.get()
    baudrate = int(self.baud_combo.get())

    if not port:
        messagebox.showwarning("Advertencia", "Selecciona un puerto COM")
        return

    try:
        # Intentar conexión
        self.log_message(f"Conectando a {port}...")
        self.ser = serial.Serial(port, baudrate, timeout=1)
        time.sleep(2)

        # Conectar directamente
        self.connected = True
        self.btn_connect.config(text="Desconectar")
        self.status_label.config(text=f"Conectado a {port}", foreground="green")
        self.btn_start.config(state="normal")
        self.log_message(f"Conectado a {port} a {baudrate} baudios")

    except Exception as e:
        self.log_message(f"Error: {str(e)}")
        messagebox.showerror("Error de Conexión", str(e))

def disconnect_serial(self):
    """Desconectar del puerto serial"""
    if self.recording:
        self.stop_recording()

    if self.ser and self.ser.is_open:
        self.ser.close()

    self.connected = False
    self.btn_connect.config(text="Conectar")
    self.status_label.config(text="Desconectado", foreground="red")
    self.btn_start.config(state="disabled")
    self.btn_stop.config(state="disabled")

```

```

        self.log_message("Desconectado del Arduino")

# =====
# FUNCIÓN: Enviar comandos al Arduino
# =====
def send_command(self, cmd):
    """Enviar comando por serial al Arduino"""
    if self.connected and self.ser:
        try:
            self.ser.write(f"{cmd}\n".encode())
            self.log_message(f"Enviado: {cmd}")
        except Exception as e:
            self.log_message(f"Error enviando comando: {e}")

def send_m1(self):
    """Enviar ángulo al Motor 1"""
    try:
        angle = float(self.entry_m1.get())
        self.send_command(f"M1:{angle}")
    except ValueError:
        messagebox.showerror("Error", "Ingresa un número válido para Motor 1")

def send_m2(self):
    """Enviar ángulo al Motor 2"""
    try:
        angle = float(self.entry_m2.get())
        self.send_command(f"M2:{angle}")
    except ValueError:
        messagebox.showerror("Error", "Ingresa un número válido para Motor 2")

def send_sss(self):
    """Enviar ángulo al Servo SSS"""
    try:
        angle = int(self.entry_sss.get())
        if 0 <= angle <= 180:
            self.send_command(f"SSS:{angle}")
        else:
            messagebox.showerror("Error", "El ángulo debe estar entre 0 y 180")
    except ValueError:
        messagebox.showerror("Error", "Ingresa un número válido para Servo SSS")

def send_sis(self):
    """Enviar ángulo al Servo SIS"""
    try:
        angle = int(self.entry_sis.get())
        if 0 <= angle <= 180:
            self.send_command(f"SIS:{angle}")
        else:
            messagebox.showerror("Error", "El ángulo debe estar entre 0 y 180")
    except ValueError:
        messagebox.showerror("Error", "Ingresa un número válido para Servo SIS")

# =====

```

```

# FUNCIÓN: Control de adquisición de datos
# =====
def start_recording(self):
    """Iniciar grabación de datos a archivo CSV"""
    if not self.connected:
        messagebox.showwarning("Advertencia", "Conéctate al Arduino primero")
        return

    # Pedir ubicación para guardar archivo
    filename = filedialog.asksaveasfilename(
        defaultextension=".csv",
        filetypes=[("CSV files", "*.csv"), ("All files", "*.*")],
        initialfile=f"datos_serial_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    )

    if not filename: # Usuario canceló
        return

    try:
        # Crear archivo CSV
        self.csv_file = open(filename, 'w', newline="", encoding='utf-8')
        self.csv_writer = csv.writer(self.csv_file)
        self.sample_count = 0
        self.start_timestamp = None

        # **PASO 1: Solicitar header al Arduino**
        self.log_message("Solicitando estructura de datos al Arduino...")

        # Limpiar buffer de entrada antes de solicitar header
        if self.ser and self.ser.in_waiting:
            self.ser.reset_input_buffer()

        # Enviar comando HEADER
        self.send_command("HEADER")

        # Esperar respuesta del header (con timeout)
        header_received = False
        max_wait_time = 3 # segundos
        start_time = time.time()

        while time.time() - start_time < max_wait_time:
            if self.ser and self.ser.in_waiting:
                try:
                    line = self.ser.readline().decode('utf-8', errors='ignore').strip()

                    if line.startswith("Timestamp,"):
                        # Guardar header en archivo CSV como primera línea
                        self.csv_header = line.split(',')
                        self.csv_writer.writerow(self.csv_header)
                        self.csv_file.flush()
                        header_received = True
                        self.log_message(f"✓ Header recibido y guardado")
                        self.log_message(f"Columnas: {len(self.csv_header)}")

```

```

        break
    elif line: # Otro mensaje mientras esperamos
        self.log_message(f"Mientras esperaba header: {line}")

    except Exception as e:
        self.log_message(f"Error leyendo header: {e}")

    time.sleep(0.01) # Pequeña pausa para no saturar CPU

if not header_received:
    self.log_message("⚠ No se recibió header del Arduino")
    self.log_message("Usando estructura por defecto...")

    # Header por defecto basado en el código Arduino
    default_header = [
        "Timestamp", "Pos1", "PWM1", "Pos2", "PWM2",
        "SSS", "SIS", "rho", "rho_real", "theta", "theta_real", "gamma", "diametro"
    ]
    self.csv_header = default_header
    self.csv_writer.writerow(default_header)
    self.csv_file.flush()
    self.log_message(f"✓ Header por defecto guardado")

# **PASO 2: Iniciar grabación de datos**
self.log_message("Iniciando adquisición de datos...")

# Cambiar estado
self.recording = True
self.recording_label.config(text="Grabando...", foreground="green")
self.btn_start.config(state="disabled")
self.btn_stop.config(state="normal")

# Enviar comando START al Arduino para comenzar envío de datos
self.send_command("START")
self.send_m1()
self.send_m2()
self.send_sss()
self.send_sis()

# Información adicional
file_info = f"Archivo: {filename}"
if hasattr(self, 'csv_header'):
    file_info += f"\nColumnas: {len(self.csv_header)}"

self.log_message(f"✓ Grabación iniciada")
self.log_message(file_info)

# Actualizar etiqueta del archivo
short_filename = filename.split('/')[-1] if '/' in filename else filename
short_filename = short_filename.split('\\')[-1] if '\\' in filename else short_filename
self.recording_label.config(text=f"Grabando: {short_filename[:20]}...")

except PermissionError:

```



```

except Exception as e:
    self.log_message(f"Error en lectura serial: {e}")

time.sleep(0.01) # Pequeña pausa para no saturar CPU

def process_serial_line(self, line):
    """Procesar línea recibida del Arduino"""
    # Si es un ACK o mensaje de estado
    if line.startswith("ACK_") or line.startswith("ARDUINO_READY"):
        self.log_message(f"Arduino: {line}")

    # **MODIFICAR: Si es el header, solo registrar pero NO guardar (ya se guardó en
start_recording)**
    elif line.startswith("Timestamp,"):
        # Solo registrar que se recibió el header
        if not hasattr(self, 'csv_header') or not self.csv_header:
            self.csv_header = line.split(',')
            self.log_message(f"Header recibido: {line}")
        else:
            self.log_message(f"Header recibido (ya guardado): {line}")

    # Si es una línea de datos CSV y estamos grabando
    elif ',' in line and self.recording and hasattr(self, 'csv_header'):
        # **AGREGAR: Verificar que no sea el header repetido**
        if line.startswith("Timestamp,"):
            self.log_message("Advertencia: Header repetido recibido, ignorando")
            return

    try:
        # Separar valores
        values = line.split(',')

        # **AGREGAR ESTO: Ajustar timestamp si está activado**
        if self.ts_relativo_var.get() and len(values) > 0:
            try:
                current_ts = int(values[0])
                if self.sample_count == 0: # Primera muestra
                    self.start_timestamp = current_ts
                    values[0] = "0" # Primera muestra = 0
                else:
                    # Calcular tiempo relativo
                    values[0] = str(current_ts - self.start_timestamp)
            except ValueError:
                pass # Si no es un número, dejarlo como está

        # Escribir en archivo CSV
        self.csv_writer.writerow(values)
        self.csv_file.flush() # Forzar escritura inmediata

        # Actualizar contador
        self.sample_count += 1
        self.root.after(0, self.update_sample_count)

```

```

        except Exception as e:
            self.log_message(f"Error procesando datos: {e}")

    # Cualquier otra línea
    else:
        self.log_message(f"Recibido: {line}")

# =====
# FUNCIÓN: Utilidades de interfaz
# =====
def update_sample_count(self):
    """Actualizar contador de muestras en la interfaz"""
    self.sample_label.config(text=f"Muestras: {self.sample_count}")

def log_message(self, message):
    """Agregar mensaje a la consola"""
    timestamp = datetime.now().strftime("%H:%M:%S")
    self.console.insert(tk.END, f"[{timestamp}] {message}\n")
    self.console.see(tk.END) # Auto-scroll

    # Limitar tamaño de consola
    lines = self.console.get(1.0, tk.END).split("\n")
    if len(lines) > 100:
        self.console.delete(1.0, "2.0")

# =====
# FUNCIÓN AGREGADA: Cerrar aplicación
# =====
def on_closing(self):
    """Manejar cierre de la aplicación - FUNCIÓN FALTANTE"""
    if self.recording:
        self.stop_recording()

    if self.connected:
        self.disconnect_serial()

    self.root.destroy()

# =====
# PROGRAMA PRINCIPAL
# =====
def main():
    # Crear ventana principal
    root = tk.Tk()

    # Crear aplicación
    app = SimpleRobotControl(root)

    # Configurar cierre seguro
    root.protocol("WM_DELETE_WINDOW", app.on_closing)

    # Ejecutar aplicación

```

```
root.mainloop()
```

```
if __name__ == "__main__":  
    main()
```